

Unit 3

(1) What are the steps associated with the knowledge Engineering process?

Discuss them by applying the steps to any real world application of your choice.

Knowledge Engineering

The general process of constructing knowledge base is called knowledge engineering. A knowledge engineer is someone who investigates a particular domain, learns what concepts are important in that domain, and creates a formal representation of the objects and relations in the domain. We will illustrate the knowledge engineering process in an electronic circuit domain that should already be fairly familiar,

The steps associated with the knowledge engineering process are :

1 Identify the task.

The task will determine what knowledge must be represented in order to connect problem instances to answers. This step is analogous to the PEAS process for designing agents.

2. Assemble the relevant knowledge. The knowledge engineer might already be an expert in the domain, **or** might need to work with real experts to extract what they know-a process called **knowledge acquisition**.

3. Decide on a vocabulary of predicates, functions, and constants. That is, translate the important domain-level concepts into logic-level names. Once the choices have been made, the result is a vocabulary that is known as the **ontology** of the domain. The word ontology means a particular theory of the nature of being or existence.

4. Encode general knowledge about the domain. The knowledge engineer writes down the axioms for all the vocabulary terms. This pins down (to the extent possible) the meaning of the terms, enabling the expert to check the content. Often, this step reveals misconceptions or gaps in the vocabulary that must be fixed by returning to step 3 and iterating through the process.

5. Encode a description of the specific problem instance

For a logical agent, problem instances are supplied by the sensors, whereas a "disembodied" knowledge base is supplied with additional sentences in the same way that traditional programs are supplied with input data.

6. Pose queries to the inference procedure and get answers. This is where the reward is: we can let the inference procedure operate on the axioms and problem-specific facts to derive the facts we are interested in knowing.

7. Debug the knowledge base.

$\neg \text{NumOfLegs}(x,4) \Rightarrow \text{Mammal}(x)$ Is false for reptiles ,amphibians.

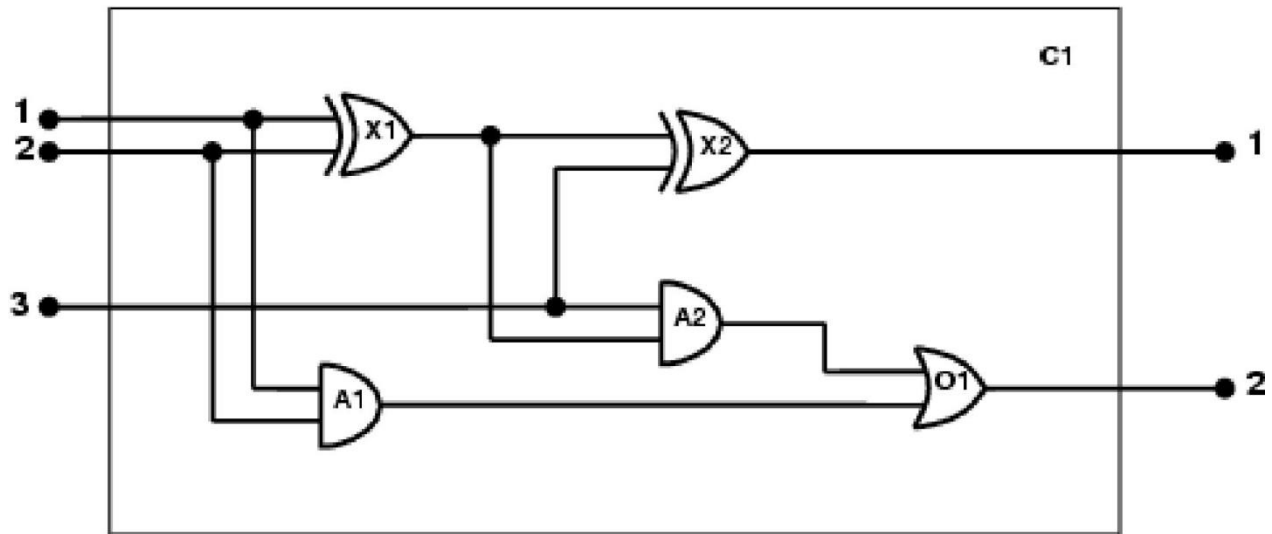
To understand this seven-step process better, we now apply it to an extended example-the domain of electronic circuits.

The electronic circuits domain

One
1

bit

adder



1 Identify the task

There are many reasoning tasks associated with digital circuits. At the highest level, one analyzes the circuit's functionality. For example, what are all the gates connected to the first input terminal? Does the circuit contain feedback loops? These will be our tasks in this section.

2 Assemble the relevant knowledge

What do we know about digital circuits? For our purposes, they are composed of wires and gates. Signals flow along wires to the input terminals of gates, and each gate produces a signal on the output terminal that flows along another wire.

3 Decide on a vocabulary

We now know that we want to talk about circuits, terminals, signals, and gates. The next step is to choose functions, predicates, and constants to represent them. We will start from individual gates and move up to circuits.

First, we need to be able to distinguish a gate from other gates. This is handled by naming gates with constants: X_1 , X_2 , and so on

Type(X_1) = XOR

Type(X_1 , XOR) –binary predicate

XOR(X_1) –individual type

A gate or circuit can have one or more terminal. For X_1 the terminals are X_1In_1 , X_1In_2 , X_1Out_1

X_1In_1 . 1st input of gate X_1

X_1In_2 -2nd input of gate X_1

X_1Out_1 .Output of gate X_1

4 Encode general knowledge of the domain

One sign that we have a good ontology is that there are very few general rules which need to be specified. A sign that we have a good vocabulary is that each rule can be stated clearly and concisely. With our example, we need only seven simple rules to describe everything we need to know about circuits:

1. If two terminals are connected, then they have the same signal:

$\Box t_1, t_2 \text{ Connected}(t_1, t_2) \Box \text{Signal}(t_1) = \text{Signal}(t_2)$

2. The signal at every terminal is either 1 or 0 (but not both):

$$\forall t \text{ Signal}(t) = 1 \vee \text{Signal}(t) = 0 \\ \neq 0$$

3. Connected is a commutative predicate:

$$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \vee \text{Connected}(t_2, t_1)$$

4. An OR gate's output is 1 if and only if any of its inputs is 1:

$$\forall g \text{ Type}(g) = \text{OR} \vee \text{Signal}(\text{Out}(1,g)) = 1 \vee \forall n \text{ Signal}(\text{In}(n,g)) = 1$$

5. An AND gate's output is 0 if and only if any of its inputs is 0:

$$\forall g \text{ Type}(g) = \text{AND} \vee \text{Signal}(\text{Out}(1,g)) = 0 \vee \forall n \text{ Signal}(\text{In}(n,g)) = 0$$

6. An XOR gate's output is 1 if and only if its inputs are different:

$$\forall g \text{ Type}(g) = \text{XOR} \vee \text{Signal}(\text{Out}(1,g)) = 1 \vee \text{Signal}(\text{In}(1,g)) \neq \text{Signal}(\text{In}(2,g))$$

7. A NOT gate's output is different from its input:

$$\forall g \text{ Type}(g) = \text{NOT} \vee \text{Signal}(\text{Out}(1,g)) \neq \text{Signal}(\text{In}(1,g))$$

5 Encode the specific problem instance

The circuit shown in Figure is encoded as circuit *C1* with the following description. First, we categorize the gates:

$$\text{Type}(X_1) = \text{XOR} \quad \text{Type}(X_2) = \text{XOR}$$

$$\text{Type}(A_1) = \text{AND}$$

$$\text{Type}(X_2) = \text{XOR}$$

$$\text{Type}(A_2) = \text{AND}$$

$$\text{Type}(O_1) = \text{OR}$$

$$\text{Connected}(\text{Out}(1,X_1), \text{In}(1,X_2))$$

$$\text{Connected}(\text{In}(1,C_1), \text{In}(1,X_1))$$

$$\text{Connected}(\text{Out}(1,X_1), \text{In}(2,A_2))$$

$$\text{Connected}(\text{In}(1,C_1), \text{In}(1,A_1))$$

$$\text{Connected}(\text{Out}(1,A_2), \text{In}(1,O_1))$$

$$\text{Connected}(\text{In}(2,C_1), \text{In}(2,X_1))$$

$$\text{Connected}(\text{Out}(1,A_1), \text{In}(2,O_1))$$

$$\text{Connected}(\text{In}(2,C_1), \text{In}(2,A_1))$$

$$\text{Connected}(\text{Out}(1,X_2), \text{Out}(1,C_1))$$

$$\text{Connected}(\text{In}(3,C_1), \text{In}(2,X_2))$$

$$\text{Connected}(\text{Out}(1,O_1), \text{Out}(2,C_1))$$

$$\text{Connected}(\text{In}(3,C_1), \text{In}(1,A_2))$$

6 Pose queries to the inference procedure

What combinations of inputs would cause the first output of *C1* (the sum bit) to be 0 and the second output of *C1* (the carry bit) to be 1?

$$\forall i_1, i_2, i_3, o_1, o_2 \text{ Signal}(\text{In}(1,C_1)) = i_1 \vee \text{Signal}(\text{In}(2,C_1)) = i_2 \vee \text{Signal}(\text{In}(3,C_1)) = i_3 \vee \\ \text{Signal}(\text{Out}(1,C_1)) = 0 \vee \text{Signal}(\text{Out}(2,C_1)) = 1$$

The answer are substitutions for the variable i_1, i_2 and i_3 such that the resulting sentence is entailed by the knowledge base, There are three such substitutions:

$\{i_1/1, i_2/1, i_3/0\} \{i_1/2, i_2/0, i_3/1\} \{i_1/0, i_2/1, i_3/1\}$

What are the possible sets of values of all the terminals for the adder circuit?

$\exists i_1, i_2, i_3, o_1, o_2 \text{ Signal(In(1, C}_1\text{))} = i_1 \wedge \text{Signal(In(2, CO))} = i_2$
 $\wedge \text{Signal(In(3, } i_1\text{))} = i_3 \wedge \text{Signal(Out(1, CO))} = o_1 \wedge \text{Signal(Out(2, CO))} = o_2$

This final query will return a complete input/output table for the device, which can be used to check that it does in fact add its inputs correctly. This is a simple example of **circuit verification**.

7 Debug the knowledge base

The knowledge base is checked with different constraints. For example if the assertion $1 \neq 0$ is not included in the knowledge base then it is variable to prove any output for the circuit ,except for the input cases 000 and 110

$\Box i_1, i_2, o \text{ Signal(In(1, C}_1\text{))} = i_1 \wedge \text{Signal(In(2, C}_1\text{))} = i_2 \wedge \text{Signal(Out(1, X}_1\text{))}$

This claim that no output are known at X_1 for the input cases 10 and 01. Therefore the axiom for XOR gate is considered

$\text{Signal(Out(1, X}_1\text{))} = 1 \wedge \text{Signal(In(1, X}_1\text{))} \neq \text{Signal(In(2, X}_1\text{))}$

If the input signal are known 1 and 0 , the above sentence reduces to $\text{Signal(Out(1, X}_1\text{))} = 1 \wedge 1 \neq 0$

Now the system is unable to infer the $\text{Signal(Out(1, X}_1\text{))} = 1$

Unification

Generalized Modus Ponens

- For atomic sentences p_i, p_i', q where there is a substitution q such that $\text{Subst}\{q, p_i'\} = \text{Subst}\{q, p_i\}$:

$$\frac{p_1', p_2', \dots, p_n', (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}$$

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$	
$\text{King}(\text{John})$	
$\text{Greedy}(\text{John})$	
$\text{Brother}(\text{Richard}, \text{John}) .$	
$p_1' \text{ is } \text{King}(\text{John})$	$p_1 \text{ is } \text{King}(x)$
$p_2' \text{ is } \text{Greedy}(y)$	$p_2 \text{ is } \text{Greedy}(x)$
$\theta \text{ is } \{x/\text{John}, y/\text{John}\}$	$q \text{ is } \text{Evil}(x)$
$\text{SUBST}(\theta, q) \text{ is } \text{Evil}(\text{John}) .$	

Unification

- It is the process of finding substitutions that make two atomic sentences identical
- Lifted inference rules require finding substitution that make different logical expression look identical this process is called unification
- It is the key component of first order inference logic algorithm

UNIFY (p,q)= θ Where SUBST(θ ,p)= SUBST(q, θ)

Θ - Unifier of two sentences

P-S1(x,x)

1(y,z)

Assume $\theta=y$

Unification algorithm returns failure as the result at two factors

- i) Two sentence have different predicate name
Ex hate(m,c)
try(m,c)
- ii) Two sentence will have different number of arguments
Ex hate(m,c,x)
hate(m,c)

Query Knows (John,x) whom does john knows

The knowledge base contain the following information

Knows(John,x)

Knows(John,Jane)

Knows(y,Bill)

Knows(y,Mother(y))

Knows(x,Eizabeth)

UNIFY(Knows(John,x), Knows(John,Jane)) = $\{x/\text{Jane}\}$

UNIFY(Knows(John,x), Knows(y,Bill)) = $\{x/\text{Bill}, y/\text{John}\}$

UNIFY(Knows(John,x), Knows(y,Mother(y))) = $\{y/\text{John}, x/\text{Mother}(\text{John})\}$

UNIFY(Knows(John,x), Knows(x,Eizabeth)) = fail .

Consider the last sentence:

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x, \text{Elizabeth})) = \text{fail}$

- This fails because x cannot take on two values
- But “Everyone knows Elizabeth” and it should not fail
- Must standardize apart one of the two sentences to eliminate reuse of variable

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(z_{17}, \text{Elizabeth})) = \{x/\text{Elizabeth}, z_{17}/\text{John}\}$

Standardizing apart eliminates overlap of variables, e.g., $\text{Knows}(z_{17}, \text{OJ})$ by renaming its variables to avoid name clashes

Most General Unifier

Multiple unifiers are possible:

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, z))$

$\{y/\text{John}, x/z\}$

or

$\{y/\text{John}, x/\text{John}, z/\text{John}\}$

Which is better, $\text{Knows}(\text{John}, z)$ or $\text{Knows}(\text{John}, \text{John})$?

- Second could be obtained from first with extra subs
- First unifier is more general than second because it places fewer restrictions on the values of the variables

There is a single most general unifier for every unifiable pair of expressions

Occur Check

$t1 = \text{likes}(X, Y)$

$t2 = \text{likes}(g(Y),$

An equation that has a variable on one side and a term containing that variable on the other side cannot be solved no matter what the substitute is. So the unification fails. The above phenomenon is called "occurs check".

Unification algorithm

function UNIFY(x, y, θ) **returns** a substitution to make x and y identical

inputs: x , a variable, constant, list, or compound

y , a variable, constant, list, or compound

θ , the substitution built up so far

if $\theta = \text{failure}$ **then return failure**

else if $x = y$ **then return** θ

else if VARIABLE?(x) **then return** UNIFY-VAR(x, y, θ)

else if VARIABLE?(y) **then return** UNIFY-VAR(y, x, θ)

else if COMPOUND?(x) **and** COMPOUND?(y) **then**

return UNIFY(ARGS[x], ARGS[y], UNIFY(OP[x], OP[y], θ))

else if LIST?(x) **and** LIST?(y) **then**

return UNIFY(REST[x], REST[y], UNIFY(FIRST[x], FIRST[y], θ))

else return failure

function UNIFY-VAR(var, x, θ) **returns** a substitution

inputs: var , a variable

x , any expression

θ , the substitution built up so far

if $\{var/val\} \in \theta$ **then return** UNIFY(val, x, θ)

else if $\{x/val\} \in \theta$ **then return** UNIFY(var, val, θ)

else if OCCUR-CHECK?(var, x) **then return failure**

else return add $\{var/x\}$ to θ

Storage and retrieval

Once the data type for sentences and terms are defined, we need to contain a set of sentences in a KB

i) **Stores(s) – store a sentence s**

ii) **Fetch(s)-returns all unifiers**

such that query q unifies with some sentence in KB.

An example is when we ask Knows(John, x) – is an instance of fetching

- The simplest way to implement STORE and FETCH is to keep all the facts in the knowledge base in one long list
- The given query UNIFY(q, s) – given query call for every sentence s in the list, requires $O(n)$ time on an n -element KB.
- Such a process is inefficient in terms of time taken because it will unify each and every statement of knowledge base
- Inefficient because we're performing so many unifies.

- Example: unify *Knows* (John, x) with *Brother* (Richard, John)
There is no need to unify
- We can avoid such unification by indexing the facts in the knowledge base
- Predicate indexing puts all the *Knows* facts in one bucket and all the *Brother* facts in another
- The bucket stored in a hash table for efficient access.
- We can large bucket problem with the help of multiple indexing
 - Might not be a win if there are lots of clauses for a particular predicate symbol
 - Consider how many people *Know* one another
- Instead index by predicate and first argument
 - Clauses may be stored in multiple buckets

Subsumption lattice

The above set of queries are said to form a collection tree called as Subsumption lattice

How to construct indices for all possible queries that unify with it

- **Example: *Employs* (AIMA.org, Richard)**

<i>Employs</i> (AIMA.org, Richard)	Does AIMA.org employ Richard?
<i>Employs</i> (x, Richard)	Who employs Richard?
<i>Employs</i> (AIMA.org, y)	Whom does AIMA.org employ?
<i>Employs</i> (x, y)	Who employs whom?

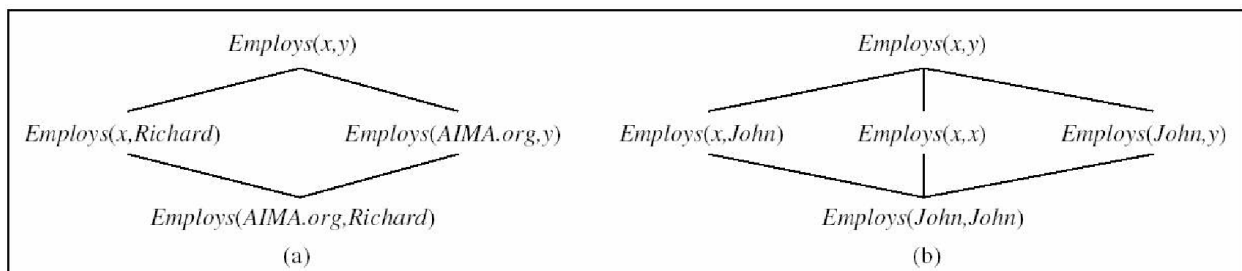


Figure 9.2 (a) The subsumption lattice whose lowest node is the sentence *Employs*(AIMA.org, Richard). (b) The subsumption lattice for the sentence *Employs*(John, John).

Properties of subsumption lattice

- The child of any node in the lattice is obtained from its parent by single substitution
- The “highest” common descendent of any two nodes is the result of applying the most general unifier
- The portion of lattice , above any ground fact can be construct
- Predicate with n arguments will create a lattice with $O(2^n)$ nodes
- Benefits of indexing may be outweighed by cost of storing and maintaining indices

8. Explain resolution concept & its procedure (NOV 2012)(NOV 2011)(MAY 2011)

- Resolution is a procedure used in proving that arguments which are expressible in predicate logic are correct
- Resolution is a procedure that produces proofs by refutation or contradiction
- Resolution leads to refute a theorem-proving technique for sentences in propositional logic and first order logic
- Resolution is a rule of inference
- Resolution is computerized theorem prover
- Resolution is so far only defined for propositional logic. The strategy is that the resolution techniques of propositional logic be adopted in predicate logic

Procedure for resolution

- Convert given proposition into clausal form
- Convert the negation of the sentence to be proved into clausal form
- Combine the clauses into a set
- Iteratively apply resolution to the set and add the resolvent to the set
- Continue until no further resolvents can be obtained or a null clause is obtained

A Predicate Logic Example

1. Marcus was a man. $man(Marcus)$
2. Marcus was a Pompeian. $Pompeian(Marcus)$
3. All Pompeians were Romans. $\forall x: Pompeian(x) \rightarrow Roman(x)$
4. Caesar was a ruler. $ruler(Caesar)$
5. All Romans were either loyal to Caesar or hated him.
 $\forall x: Roman(x) \rightarrow loyalto(x, Caesar) \vee hate(x, Caesar)$
6. Everyone is loyal to someone. $\forall x: \exists y: loyalto(x, y)$
7. People only try to assassinate rulers they aren't loyal to.
 $\forall x: \forall y: person(x) \wedge ruler(y) \wedge tryassassinate(x, y) \rightarrow \neg loyalto(x, y)$
8. Marcus tried to assassinate Caesar.
 $tryassassinate(Marcus, Caesar)$
9. All men are people. $\forall x: man(x) \rightarrow person(x)$

Conversion to Clause Form

✱ Problem:

$\forall x: [Roman(x) \wedge know(x, Marcus)] \rightarrow [hate(x, Caesar) \vee (\forall y: \exists z: hate(y, z) \rightarrow thinkcrazy(x, y))]$

✱ Solution:

■ Flatten

■ Separate out quantifiers

✱ Conjunctive Normal Form: $\neg Roman(x) \vee \neg know(x, Marcus) \vee hate(x, Caesar) \vee \neg hate(y, z) \vee thinkcrazy(x, z)$

✱ Clause Form

■ Conjunctive normal form

Converting FOL Sentences to CNF

1. Replace $\Leftrightarrow$ with equivalent (added):
 - convert $P \Leftrightarrow Q$ to $P \Rightarrow Q \wedge Q \Rightarrow P$
2. Replace $\Rightarrow$ with equivalent: convert $P \Rightarrow Q$ to $\neg P \vee Q$
3. Reduce scope of $\neg$ to single literals:
 - convert $\neg\neg P$ to P (DNE)
 - convert $\neg(P \vee Q)$ to $\neg P \wedge \neg Q$ (de Morgan's)
 - convert $\neg(P \wedge Q)$ to $\neg P \vee \neg Q$ (de Morgan's)
 - convert $\neg\forall x P$ to $\exists x \neg P$
 - convert $\neg\exists x P$ to $\forall x \neg P$
4. Standardize variables apart:
 - each quantifier must have a unique variable name
 - avoids confusion in steps 5 and 6
 - e.g. convert $\forall x P \vee \exists x Q$ to $\forall x P \vee \exists y Q$
5. Eliminate existential quantifiers (Skolemize):
 - convert $\exists x P(x)$ to $P(C)$ (EE)
C must be a new constant (Skolem constant)
 - convert $\forall x, y \exists z P(x, y, z)$ to $\forall x, y P(x, y, F(x, y))$
F() must be a new function (Skolem function) with arguments that are all enclosing universally quantified variables

Skolem Functions in FOL

Objective

- Want all variables universally quantified
- Notational variant of FOL w/o existentials
- Retain implicitly full FOL expressiveness

Skolem's Theorem

Every existentially quantified variable can be replaced by a unique *Skolem function* whose arguments are all the universally quantified variables on which the existential depends, without changing FOL.

Examples

- “Everybody likes something”

$\forall(x) \exists(y) [Person(x) \ \& \ Likes(x,y)]$

$\forall(x) [Person(x) \ \& \ Likes(x, S1(x))]$

Where $S1(x)$ = “that which x likes”

- “Every philosopher writes at least one book”

$\forall(x) \exists(y) [Philosopher(x) \ \& \ Book(y) \Rightarrow Write(x,y)]$

$\forall(x) [(Philosopher(x) \ \& \ Book(S2(x))) \Rightarrow Write(x,S2(x))]$

6. Drop quantifiers:
 - all variables are only universally quantified after step 5
 - e.g. convert $\forall x P(x) \vee \forall y Q(y)$ to $P(x) \vee Q(y)$
 - all variables in KB will be assumed to be universally quantified
7. Distribute $\wedge$ over $\vee$ to get conjunction of disjunctions :
 - convert $(P \wedge Q) \vee R$ to $(P \vee R) \wedge (Q \vee R)$

8. Flatten nested conjunctions and disjunctions:

- convert $(P \wedge Q) \wedge R$ to $(P \wedge Q \wedge R)$
- convert $(P \vee Q) \vee R$ to $(P \vee Q \vee R)$

9. Separate each conjunct (added)

split at $\wedge$'s so each conjunct is now a CNF clause

$(\neg \text{Above}(x, y) \vee \text{OnTop}(x, y) \vee \text{OnTop}(x, F(x, y))) \wedge$
 $(\neg \text{Above}(x, y) \vee \text{OnTop}(x, y) \vee \text{Above}(F(x, y), y))$

becomes:

$\neg \text{Above}(x, y) \vee \text{OnTop}(x, y) \vee \text{OnTop}(x, F(x, y))$
 $\neg \text{Above}(x, y) \vee \text{OnTop}(x, y) \vee \text{Above}(F(x, y), y)$

10. Standardize variables apart in each clause (added)

- each clause in KB must contain unique variable names
- now during unification the standardize apart step need only be done on deduced clauses (i.e. resolvents)

$\neg \text{Above}(x, y) \vee \text{OnTop}(x, y) \vee \text{OnTop}(x, F(x, y))$
 $\neg \text{Above}(x, y) \vee \text{OnTop}(x, y) \vee \text{Above}(F(x, y), y)$

Examples of Conversion to Clause Form

■ Example:

$\forall x: [\text{Roman}(x) \wedge \text{know}(x, \text{Marcus})] \rightarrow [\text{hate}(x, \text{Caesar}) \vee (\forall y: \exists z: \text{hate}(y, z) \rightarrow \text{thinkcrazy}(x, y))]$

■ Eliminate $\rightarrow$

$\forall x: \neg [\text{Roman}(x) \wedge \text{know}(x, \text{Marcus})] \vee [\text{hate}(x, \text{Caesar}) \vee (\forall y:$

$\exists \neg z: \text{hate}(y,z) \vee \text{thinkcrazy}(x,y))]$
 ■ Reduce scope of $\neg$.
 $\forall x: [\neg \text{Roman}(x) \vee \neg \text{know}(x, \text{Marcus})] \vee [\text{hate}(x, \text{Caesar}) \vee (\forall y: \forall z: \neg \text{hate}(y,z) \vee \text{thinkcrazy}(x,y))]$
 ■ "Standardize" variables:
 $\forall x: P(x) \vee \forall x: Q(x)$ converts to $\forall x: P(x) \vee \forall y: Q(y)$
 ■ Move quantifiers. $\forall x: \forall y: \forall z: [\neg \text{Roman}(x) \vee \neg \text{know}(x, \text{Marcus})] \vee [\text{hate}(x, \text{Caesar}) \vee (\neg \text{hate}(y,z) \vee \text{thinkcrazy}(x,y))]$
 ■ Eliminate existential quantifiers.
 $\exists y: \text{President}(y)$ will be converted to $\text{President}(S1)$
 $\forall x: \exists y: \text{father-of}(y,x)$ will be converted to $\forall x: \text{father-of}(S2(x),x)$
 ■ Drop the prefix.
 $[\neg \text{Roman}(x) \vee \neg \text{know}(x, \text{Marcus})] \vee [\text{hate}(x, \text{Caesar}) \vee (\neg \text{hate}(y,z) \vee \text{thinkcrazy}(x,y))]$
 ■ Convert to a conjunction of disjuncts.
 $\neg \text{Roman}(x) \vee \neg \text{know}(x, \text{Marcus}) \vee \text{hate}(x, \text{Caesar}) \vee \neg \text{hate}(y,z) \vee \text{thinkcrazy}(x,y)$

The Basis of Resolution and Herbrand's Theorem

■ Given:

$\text{winter} \vee \text{summer}$

$\neg \text{winter} \vee \text{cold}$

We can conclude:

$\text{summer} \vee \text{cold}$

■ Herbrand's Theorem:

To show that a set of clauses S is unsatisfiable, it is necessary to consider only interpretations over a particular set, called the *Herbrand universe* of S . A set of clauses S is unsatisfiable if and only if a finite subset of ground instances (in which all bound variables have had a value substituted for them) of S is unsatisfiable.

Algorithm: Propositional Resolution

1. Convert all the propositions of F to clause form.
2. Negate P and convert the result to clause form. Add it to the set of clauses obtained in step 1.
3. Repeat until either a contradiction is found or no progress can be made:
 - a) Select two clauses. Call these the parent clauses.
 - b) Resolve them together.

The *resolvent* will be the disjunction of all of the literals of both of the parent clauses with the following exception: If there are any pairs of literals L and $\neg L$ such that one of the parent clauses contains L and the other contains $\neg L$, then select one such pair and eliminate both L and $\neg L$ from the resolvent.

c) If the resolvent is the empty clause, then a contradiction has been found. If it is not, then add it to the set of clauses available to the procedure.

A Few Facts in Propositional Logic

Given Axioms

Clause Form

$$P \quad P \quad (1)$$

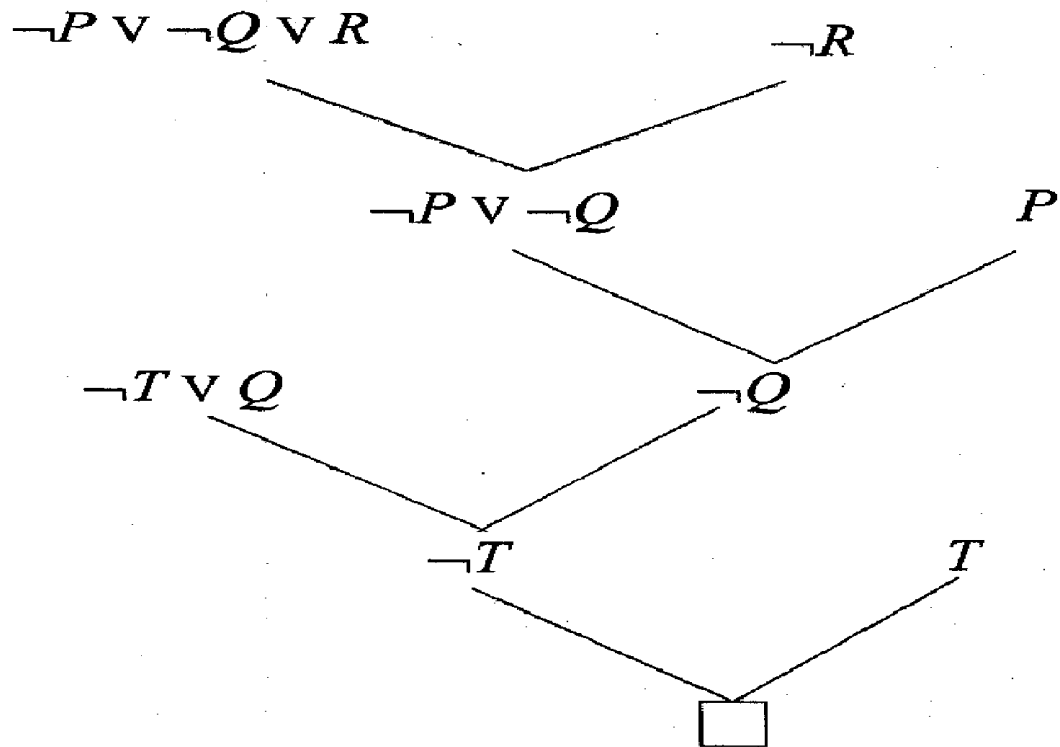
$$(P \wedge Q) \rightarrow R \quad \neg P \vee \neg Q \vee R \quad (2)$$

$$(S \vee T) \rightarrow Q \quad \neg S \vee Q \quad (3)$$

$$\neg T \vee Q \quad (4)$$

$$T \quad T \quad (5)$$

Resolution in Propositional Logic



⌘ Axioms in clause form:

1. $man(Marcus)$
2. $Pompeian(Marcus)$
3. $\neg Pompeian(x_1) \vee Roman(x_1)$
4. $Ruler(Caesar)$
5. $\neg Roman(x_2) \vee loyalto(x_2, Caesar) \vee hate(x_2, Caesar)$
6. $loyalto(x_3, f_1(x_3))$
7. $\neg person(x_4) \vee \neg ruler(y_1) \vee \neg tryassassinate(x_4, y_1) \vee$
 $loyalto(x_4, y_1)$
8. $tryassassinate(Marcus, Caesar)$

9 $Man(x_5) \vee person(x_5)$

$\exists x \text{ Owns}(\text{Nono}, x) \wedge \text{Missile}(x)$:

$\text{Owns}(\text{Nono}, M_1)$ and $\text{Missile}(M_1)$

- All of its missiles were sold to it by Colonel West

$\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$

- We also need to know that missiles are weapons

$\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

- and we must know that an enemy of America counts as “hostile”

$\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$

- West, who is American”

$\text{American}(\text{West})$

Nono, is a nation

Nation(nono)

- The country Nono, an enemy of America

$\text{Enemy}(\text{Nono}, \text{America})$

America is a nation

Nation (America)

```

function FOL-FC-ASK( $KB, \alpha$ ) returns a substitution or false

  repeat until new is empty
     $new \leftarrow \{ \}$ 
    for each sentence  $r$  in  $KB$  do
       $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-APART}(r)$ 
      for each  $\theta$  such that  $(p_1 \wedge \dots \wedge p_n)\theta = (p'_1 \wedge \dots \wedge p'_n)\theta$ 
        for some  $p'_1, \dots, p'_n$  in  $KB$ 
           $q' \leftarrow \text{SUBST}(\theta, q)$ 
          if  $q'$  is not a renaming of a sentence already in  $KB$  or new then do
            add  $q'$  to new
             $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
            if  $\phi$  is not fail then return  $\phi$ 
      add new to  $KB$ 
  return false

```

Analyse the algorithm

Sound

- Does it only derive sentences that are entailed?
- Yes, because only Modus Ponens is used and it is sound

Complete

- Does it answer every query whose answers are entailed by the KB?
- Yes if the clauses are definite clauses

Assume KB only has sentences with no function symbols

- What's the most number of iterations through algorithm?
- Depends on the number of facts that can be added
 - Let k be the arity, the max number of arguments of any predicate and
 - Let p be the number of predicates
 - Let n be the number of constant symbols
- At most pn^k distinct ground facts
- Fixed point is reached after this many iterations
- A proof by contradiction shows that the final KB is complete

Three sources of complexity

- inner loop requires finding all unifiers such that premise of rule unifies with facts of database
 - this “pattern matching” is expensive
- must check every rule on every iteration to check if its premises are satisfied
- many facts are generated that are irrelevant to goal

Step1:

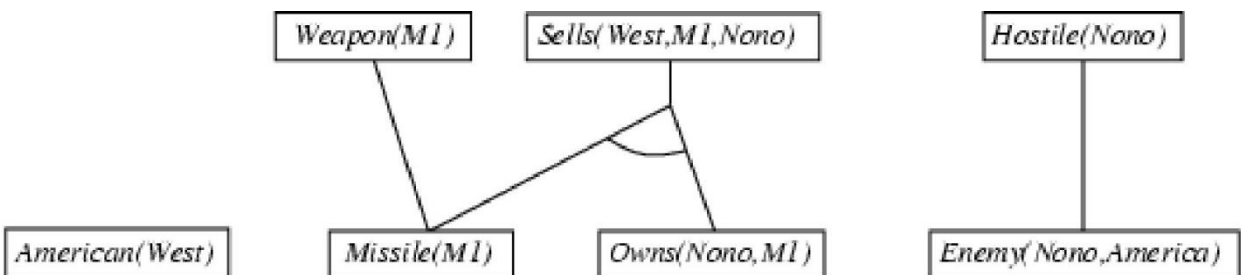
American(West)

Missile(M1)

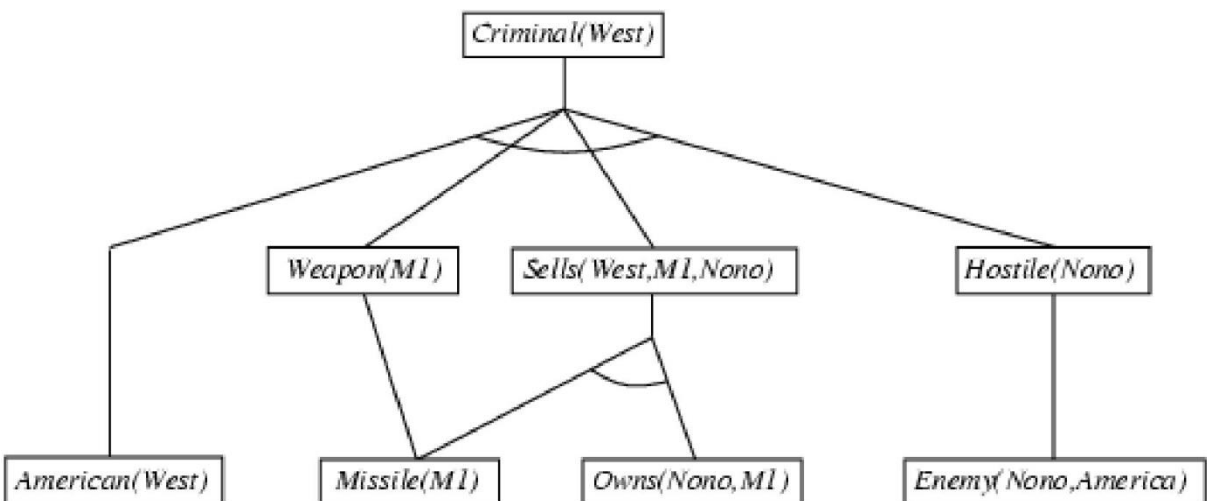
Owns(Nono,M1)

Enemy(Nono,America)

Step2:



Step3:



Efficiency of forward chaining

- ❖ Incremental forward chaining: no need to match a rule on iteration k if a premise wasn't added on iteration $k-1$
 - match each rule whose premise contains a newly added positive literal
- ❖ Matching itself can be expensive:
- ❖ Database indexing allows $O(1)$ retrieval of known facts

e.g., query $Missile(x)$ retrieves $Missile(M_1)$

Forward chaining is widely used in deductive databases

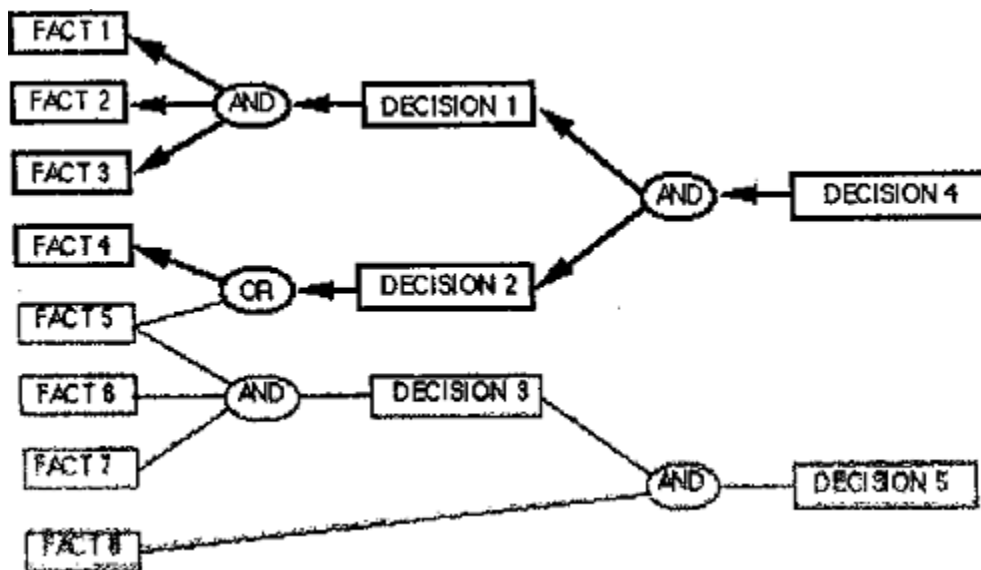
Backward chaining

In backward chaining, we start from a conclusion, which is the hypothesis we wish to prove, and we aim to show how that conclusion can be reached from the rules and facts in the data base.

To determine if a decision should be made, work backwards looking for justifications for the decision,

The conclusion we are aiming to prove is called a goal, and the reasoning in this way is known as goal-driven.

Eventually, a decision must be satisfied by facts



```

function FOL-BC-ASK( $KB, goals, \theta$ ) returns a set of substitutions
  inputs:  $KB$ , a knowledge base
             $goals$ , a list of conjuncts forming a query
             $\theta$ , the current substitution, initially the empty substitution  $\{ \}$ 
  local variables:  $ans$ , a set of substitutions, initially empty

  if  $goals$  is empty then return  $\{ \theta \}$ 
   $q' \leftarrow \text{SUBST}(\theta, \text{FIRST}(goals))$ 
  for each  $r$  in  $KB$  where  $\text{STANDARDIZE-APART}(r) = (p_1 \wedge \dots \wedge p_n \Rightarrow q)$ 
    and  $\theta' \leftarrow \text{UNIFY}(q, q')$  succeeds
     $ans \leftarrow \text{FOL-BC-ASK}(KB, [p_1, \dots, p_n | \text{REST}(goals)], \text{COMPOSE}(\theta, \theta')) \cup ans$ 
  return  $ans$ 

```

The algorithm uses composition of substitution. $\text{COMPOSE}(\theta_1, \theta_2)$ is the substitution whose effect is identical to the effect of applying each substitution is

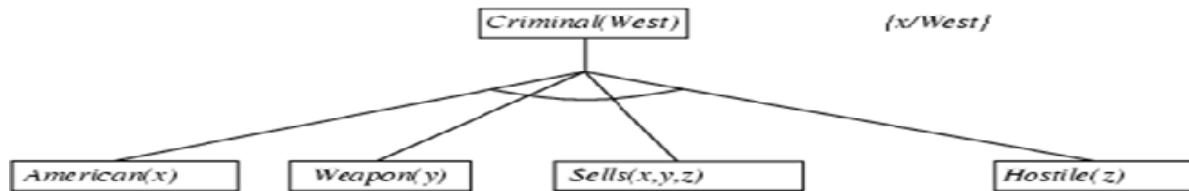
$\text{SUBST}(\text{COMPOSE}(\theta_1, \theta_2), p) = \text{SUBST}(\theta_2, \text{SUBST}(\theta_1, p))$

In the algorithm, the current variable binding, which are stored in θ , are composed with the binding resulting from unifying the goal with the clause head, given a new set of current bindings for the recursive call.

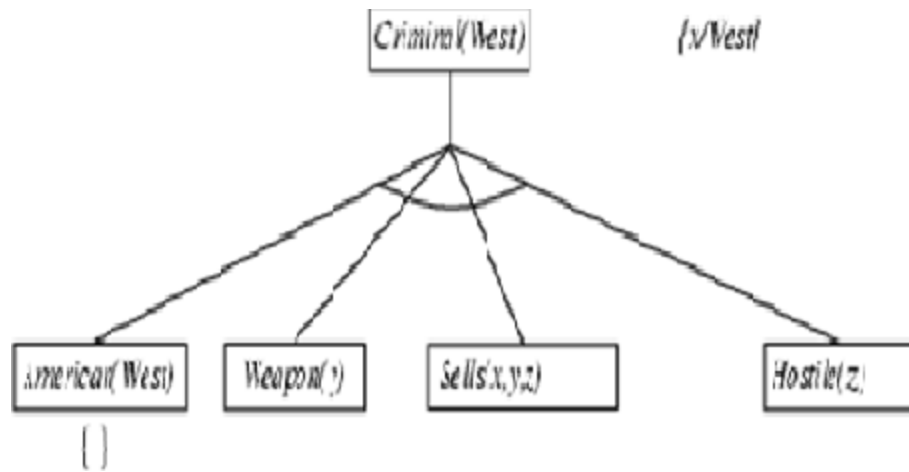
Step1

Criminal(West)

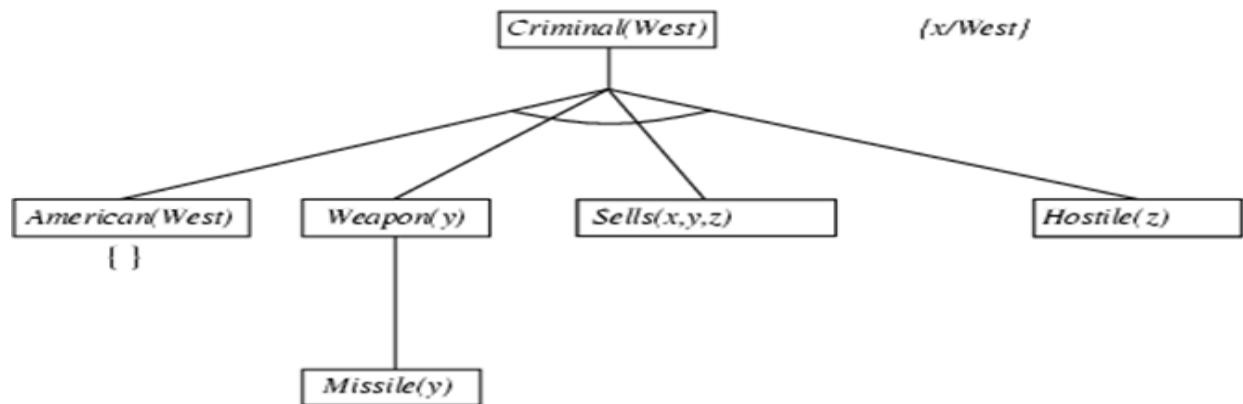
Step2



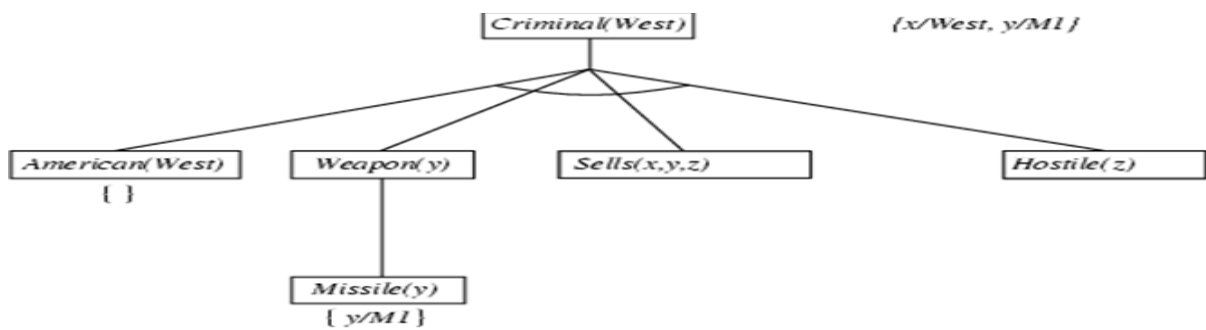
Step 3



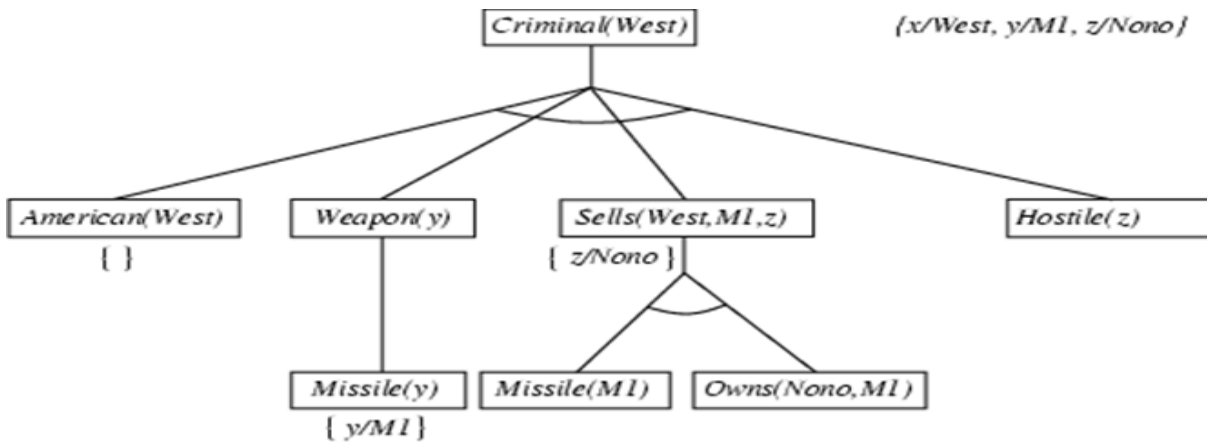
Step 4



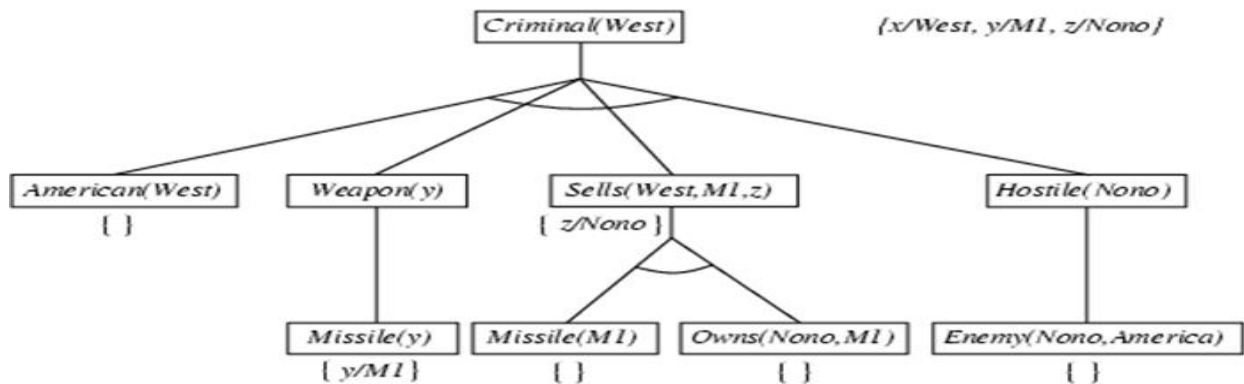
Step 5



Step 6



Step 7



properties of backward chaining

Depth-first recursive proof search: space is linear in size of proof

Incomplete due to infinite loops □ fix by checking current goal against every goal on stack

Inefficient due to repeated subgoals (both success and failure)

□ fix using caching of previous results (extra space)

Widely used for logic programming: prolog

Two marks

1 What is Ontological Engineering

Ontology refers to organizing everything in the world into hierarchy of categories. Representing the abstract concepts such as Actions, Time, Physical Objects, and Beliefs is called Ontological Engineering

2 Define diagnostic rules with example?

Diagnostic rules are used in first order logic for inferencing. The diagnostic rules generate hidden causes from observed effect. They help to deduce hidden facts in the world. For example considering the wumpum world.

The diagnostic rule finding 'pit' is,

"If square is breezy some adjacent square must contain pit", which is written as,

$$\forall x \text{Breezy}(s) \Rightarrow \exists r \text{Adjacent}(r,s) \wedge \text{pit}(r)$$

Define the first order definite clause? (NOV 2013)

First-order definite clauses closely resemble propositional definite clauses, they are disjunctions of literals of which *exactly one is positive*. A definite clause either is atomic or is an implication whose antecedent is a conjunction of positive literals and whose consequent is a single positive literal. The following are first-order definite clauses:

King(x) $\wedge$ Greedy(x) $\Rightarrow$ Evil(x) .

King(John) .

Greedy(y) .

4

S. No	Propositional logic	Predicate logic
1	It is less expressive power	It is more expressive power
2	It cannot represent relationship among objects	It represent relationship among objects
3	It does not use quantifiers	It use quantifiers
4	It does not consider generalization of objects	It consider generalization of objects